

A decorative background featuring a network diagram with nodes and connecting lines. The nodes are represented by small circles, some of which are highlighted with blue outlines or filled with blue. The lines are thin and gray, creating a web-like structure that spans the entire page.

Cryptography and Applications

Encryption/Decryption

Transform your data (**plain text**) into unrecognizable/unreadable form (**cipher text**) using auxiliary data (**keys**)

- Encryption: $\text{Enc}(\text{Plain Text}, \text{KeyE}) \rightarrow \text{Cipher Text}$
- Decryption: $\text{Dec}(\text{Cipher Text}, \text{KeyD}) \rightarrow \text{Plain Text}$

Encryption/Decryption: Assumptions

- Attacker knows the transformations (Enc, Dec).
 - No security through obscurity.
- Only key is unknown.

Encryption/Decryption: Example 1

- Enc: Reverse the plain text
 - Purdue -> eudruP
- Dec: Reverse the cipher text
 - eudruP -> Purdue

Is this good?

Encryption/Decryption: Example 2

- Enc (Caesar with key 3):
 - Purdue -> Sxugxh
- Dec (Caesar with key 3):
 - Sxugxh -> Purdue

Is this good?

Encryption/Decryption: Example 3

- Enc (Xor key aGAP@SX):
 - Purdue -> 123456
- Dec (Xor key aGAP@SX):
 - 123456 -> Purdue

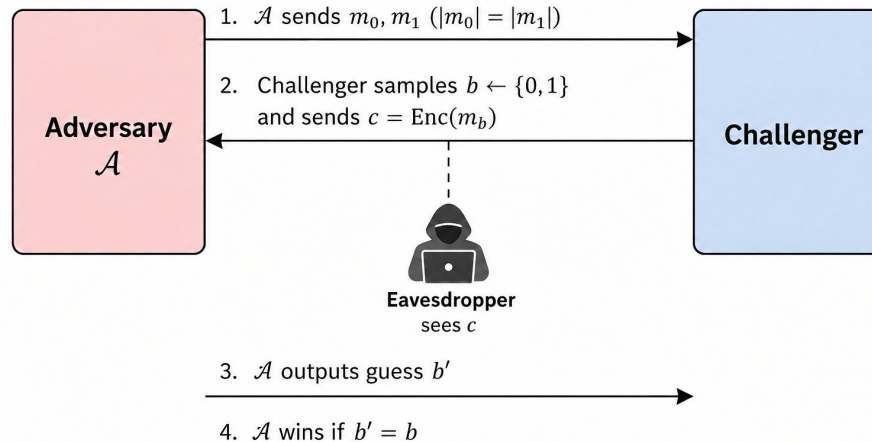
Is this good?

Necessary Properties of Enc/Dec Techniques

- Secure against eavesdropping (IND-EAV).
 - Attacker can see ciphertext
- Secure against access to encryption technique (IND-CPA).
 - Attacker can ask to encrypt arbitrary plain text.
- Secure against access to decryption technique (IND-CCA).
 - Attacker can ask to decrypt arbitrary cipher text.

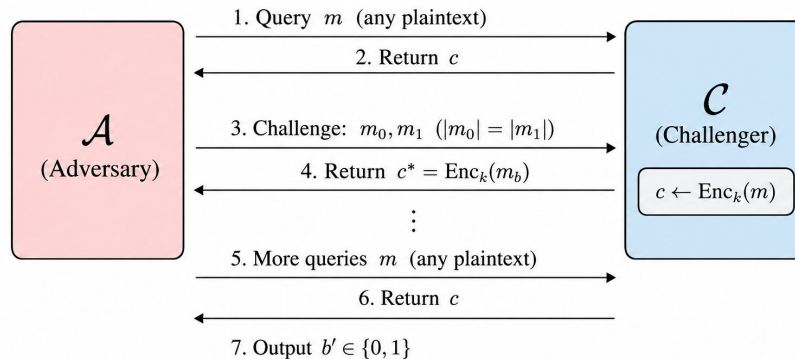
IND-EAV: Indistinguishability against eavesdropping

- Attacker can see ciphertexts but cannot guess (better than random) what challenge plain text is.



IND-CPA: Indistinguishability against chosen plaintext attack

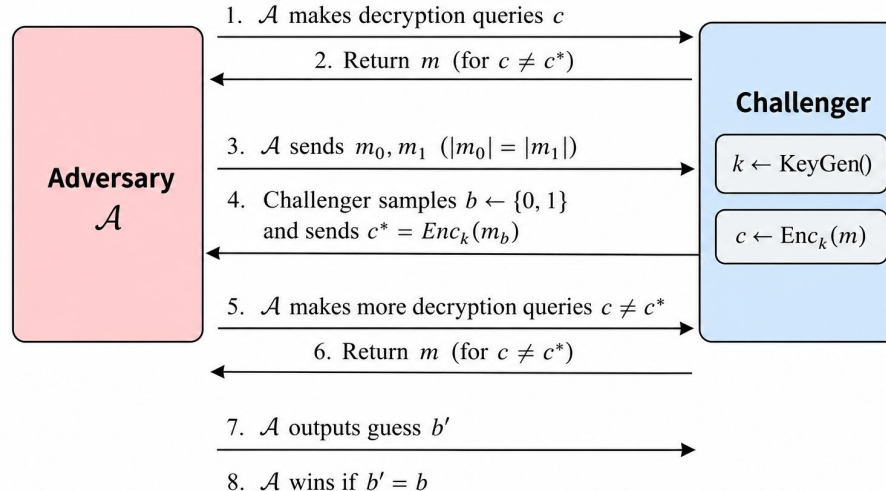
- Attacker can see ciphertext of any chosen plaintext but cannot guess (better than random) what the challenge plain text is.



$\mathcal{A}$ wins if $b' = b$.

IND-CCA: Indistinguishability against chosen ciphertext attack

- Attacker can see plaintext of any chosen ciphertext but cannot guess (better than random) what the challenge plain text is.



Necessary Properties of Enc/Dec Techniques

- IND-EAV: Should be fundamentally strong.
- IND-CPA: Should use nonce (randomize plain text every time). $\text{Enc}(\text{plain text} || \text{nonce}, \text{key})$
- IND-CCA: Any change to plain text should affect entire cipher text or Prevent creating arbitrary cipher texts.

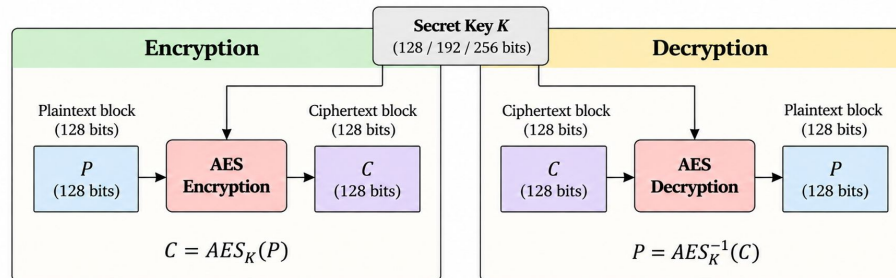
Symmetric v/s Public key crypto

- Symmetric: Same KeyE and KeyD
- Asymmetric Encryption or Public key crypto: Different KeyE (public key) and KeyD (private key)

Symmetric Encryption Algorithms

AES (Advanced Encryption Standard): Data goes through multiple rounds of transformation based on the key length.

Security depends on informal vetting, i.e., efforts trying to break it.

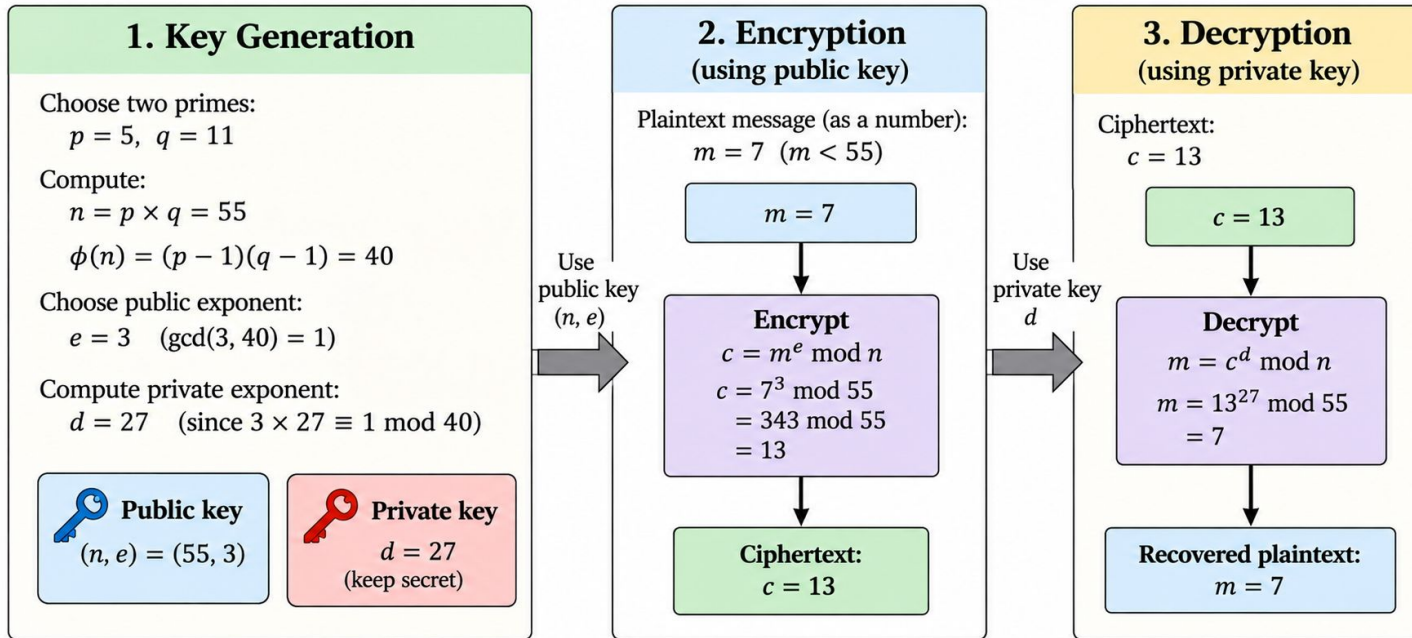


Asymmetric Encryption / Public Key Crypto

Based on hard mathematical problem: integer factorization.

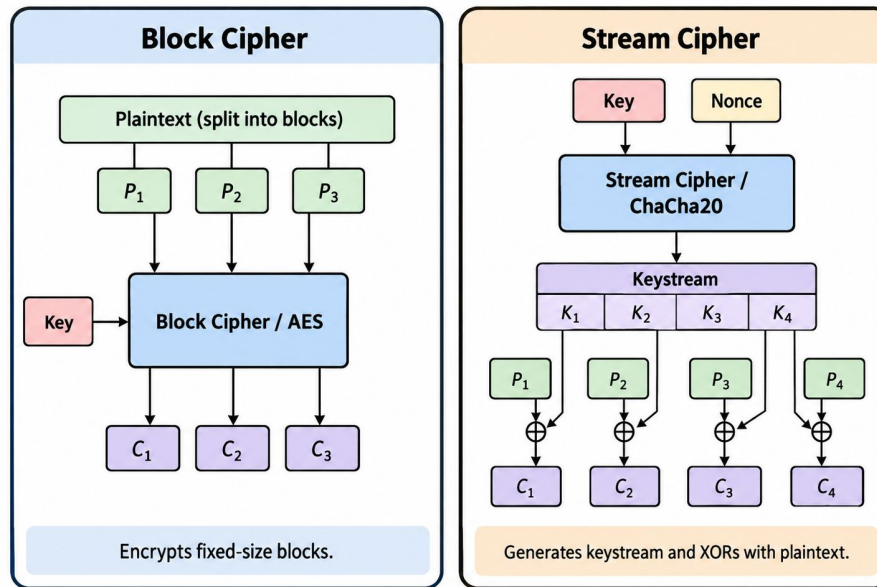
- Consider $n = p * q$ (two large prime numbers), now given n find p and q .
- No easy solution available.

RSA

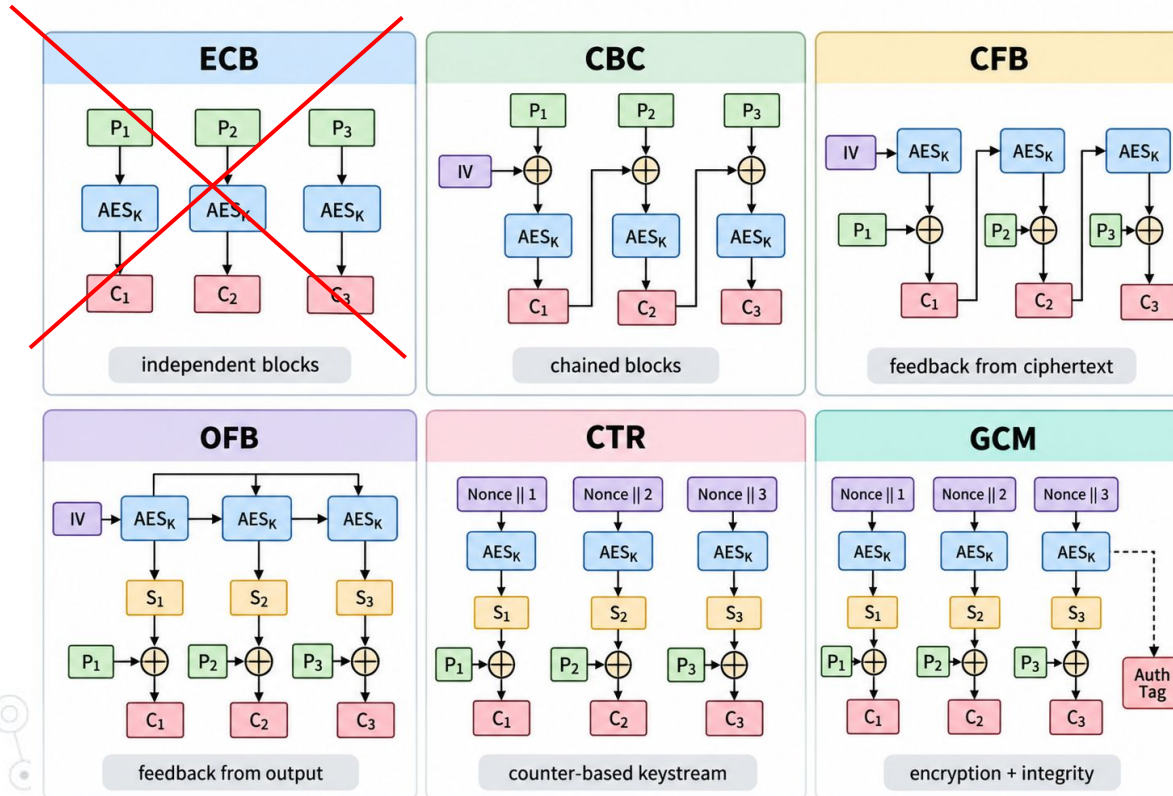


Block Cipher v/s Stream Cipher

- Block Cipher: Encrypts one block at a time.
 - Usually has hardware support.
- Stream Cipher: Can encrypt arbitrary stream of data.



Using Block Cipher for Arbitrary Data



Applications

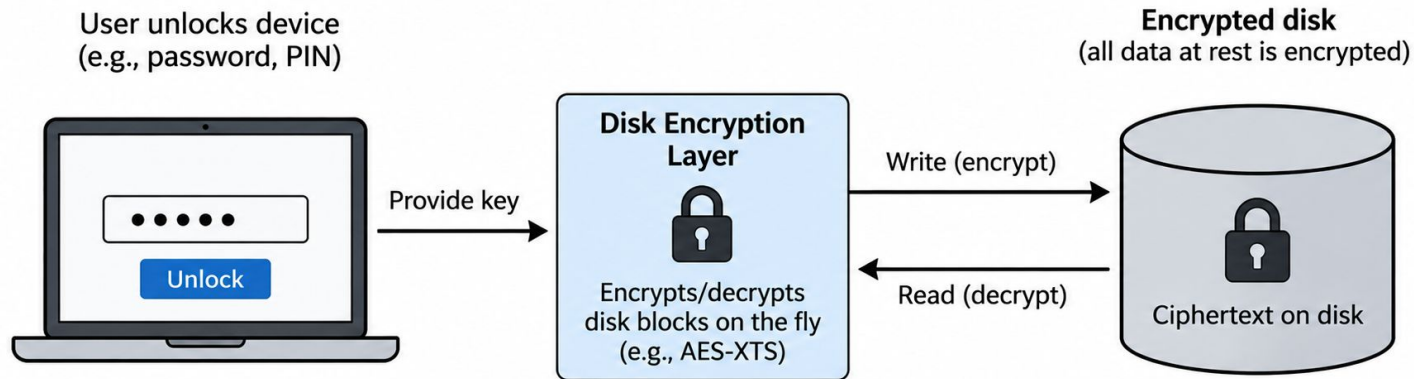
Symmetric crypto:

- Fast with specialized hardware (e.g., ARM Cryptocell).
- Requires key sharing/storage.

Suited for cases where key can be derived (i.e., avoid storage).

- E.g., Full Disk Encryption

Full Disk Encryption



Applications

Asymmetric or Public Key crypto:

- **Slow compared to symmetric crypto.**
- **No key sharing.**
- Diverse applications (encryption, signing (encryption with private key)).

Suitable to exchange information without any setup.

- E.g., Send data to Google server.
 - Encrypt using Google's public key.
 - Google can decrypt using its private key.

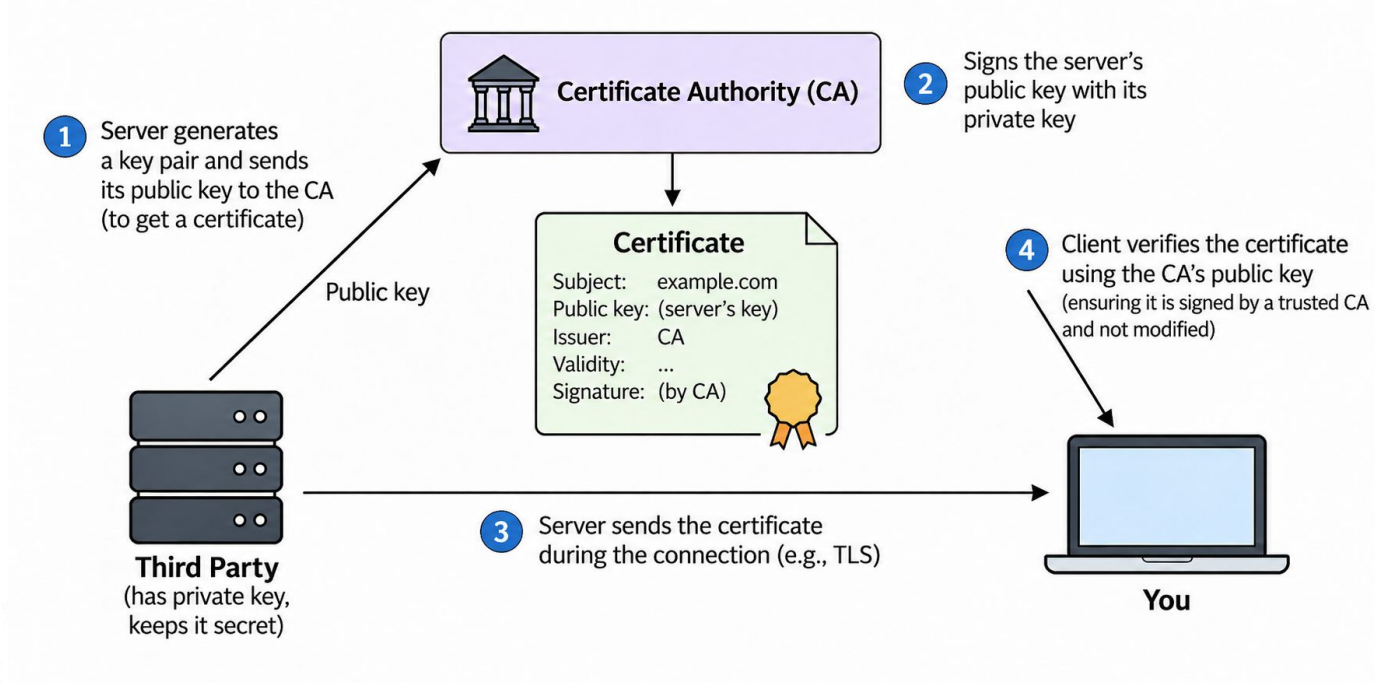
Asymmetric Crypto

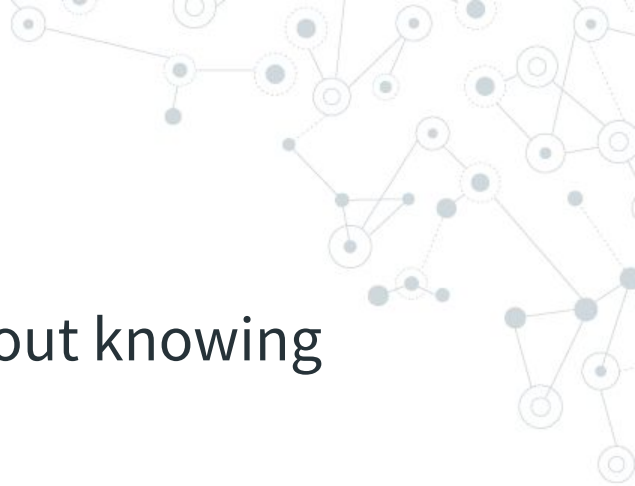
- Need Infrastructure to support public keys.
- How do we share public keys? How do we avoid attackers from spoofing them? What do we do in real world?

Asymmetric Crypto (Public Key Certificates)

- Lets rely on Trusted entities, i.e., certificate authorities (CAs).
- You have the public keys of CAs, i.e., if CA encrypt using its private key $\Rightarrow$ you can verify it.
- Assume if CA encrypts (using its private key) a message saying that “PubK is the public key of X”.

Asymmetric Crypto (Public Key Certificates)





Hashing (One way functions)

What if we want to verify some data, without knowing what the data is?

Choice 1: store the cipher text of the data!

Hashing (One way functions)

- Functions whose output changes considerably even with small changes in input.
 - $\text{Func}(A) \text{ ----- } \text{Func}(A+1)$

Secure Hashing (One way functions)

- Functions whose output changes considerably even with small changes in input.
 - $\text{Func}(A) \text{ ----- } \text{Func}(A+1)$
- Functions which are hard to inverse (pre-image resistance and fixed size output).
 - $\text{Func}(A) \rightarrow B$; Given B it is hard to know A or anything about A .
- Hard to find collisions (second pre-image resistance).
Given B (i.e., $\text{Func}(A)$), it is hard to find X , such that $X \neq A$ and $\text{Func}(X) = B$.

Hash Functions

- ~~MD5 (Don't use)~~
- SHA (Secure hash algorithm) -- 256 and above

Hash Functions: Applications

- Integrity protection (password verification):
 - Store hash instead of password.
 - Compare hash of password with stored hash.
- Verification without confidentiality:
 - Data, $\text{Encrypt}(\text{private key}, \text{hash}(\text{data}))$
 - Anyone can verify the integrity of data, e.g., Public keys